

# Azure Key Vault Secrets with ASP.NET Core

## 1. What We Will Learn

In this demo, we will learn how to use **Azure Key Vault Secrets** in an ASP.NET Core application.

We will first read a connection string from:

```
appsettings.json
```

Then we will modify the application so that the same connection string is retrieved from:

```
Azure Key Vault
```

### Overall Flow

Initial Approach:

```
ASP.NET Core Application
  |
  ↓
appsettings.json
  |
  ↓
SQL Connection String
```

After using Key Vault:

```
ASP.NET Core Application
  |
  ↓
Azure Key Vault
  |
  ↓
SQL Connection String Secret
```

## 2. What is a Secret?

A **Secret** is sensitive information that should not be exposed or hardcoded inside an application.

Examples:

- Database connection strings
- Database passwords
- API keys
- Storage account connection strings
- Client secrets
- Access tokens

For example:

```
SQL Connection String
Server=...
Database=...
User=...
Password=...
```

Instead of storing this directly in the application configuration, we can store it securely in Azure Key Vault.

---

## 3. Why Not Store Secrets in appsettings.json?

Normally, an ASP.NET Core application might contain:

```
{
  "ConnectionStrings": {
    "SQLConnectionString": "Server=...;Database=...;Password=..."
  }
}
```

The problem is that configuration files can potentially be accessed by developers, deployment systems, source-control repositories, or other users who have access to the application.

Therefore, sensitive values should preferably be stored in a dedicated secret-management service such as Azure Key Vault.

---

## 4. Demo – ASP.NET Core Application

We create an ASP.NET Core Web Application using Visual Studio.

Example project name:

AzureSecretsDemo

The application contains:

```
Controller
  ↓
Configuration
  ↓
appsettings.json
```

Initially, the application reads the SQL connection string directly from `appsettings.json`.

---

## 5. Add Connection String to appsettings.json

Open:

appsettings.json

Add a `ConnectionStrings` section.

Example:

```
{
  "ConnectionStrings": {
    "SQLConnectionString": "SQL DB connection string"
  }
}
```

The important part is the key name:

SQLConnectionString

---

## 6. Read the Connection String Using IConfiguration

In the Home Controller, inject:

```
IConfiguration
```

Example:

```
private readonly IConfiguration _configuration;

public HomeController(IConfiguration configuration)
{
    _configuration = configuration;
}
```

Now we can read the connection string:

```
var sqlConnectionString =
    _configuration.GetConnectionString("SQLConnectionString");
```

### What is happening?

The application looks for:

```
ConnectionStrings
    ↓
SQLConnectionString
    ↓
Value
```

---

## 7. Display the Value on the Web Page

For demonstration purposes, we can pass the value to the View.

Example:

```
ViewBag.SqlConnectionString = sqlConnectionString;
```

Then in the View:

```
SQL Connection String Value:  
@ViewBag.SqlConnectionString
```

When we run the application, the connection string is displayed on the web page.

This confirms that:

```
ASP.NET Core  
  ↓  
IConfiguration  
  ↓  
appsettings.json  
  ↓  
SqlConnectionString
```

is working.

---

## 8. Now Move the Secret to Azure Key Vault

We don't want the application to retrieve the connection string from:

```
appsettings.json
```

Instead, we want:

```
ASP.NET Core  
  ↓  
Azure Key Vault  
  ↓  
Secret
```

---

## 9. Create the Secret in Azure Key Vault

Open the Azure Key Vault.

Go to:

```
Key Vault
↓
Objects
↓
Secrets
```

Click:

**Generate/Import**

---

## 10. Key Vault Secret Naming

The application configuration contains:

```
ConnectionStrings
↓
SQLConnectionString
```

To map this hierarchical configuration structure to a Key Vault secret, we can use:

```
ConnectionStrings--SQLConnectionString
```

Notice that we use **two hyphens** ( `--` ).

### Example

```
ConnectionStrings--SQLConnectionString
```

This represents:

```
ConnectionStrings
↓
SQLConnectionString
```

### Important

In Azure Key Vault configuration integration, the double hyphen `--` is used as a delimiter to represent configuration hierarchy.

---

## 11. Create the Secret

Create the following secret:

```
Name :
ConnectionString--SQLConnectionString
```

Example value:

```
This secret will be fetched from Key Vault
```

Click:

**Create**

Now the secret is stored in Azure Key Vault.

---

## 12. Verify the Secret

Open the newly created secret.

Select:

**Show Secret Value**

We can see the value:

```
This secret will be fetched from Key Vault
```

This confirms that the secret exists in Key Vault.

---

## 13. Install Required NuGet Packages

The ASP.NET Core application needs packages to integrate with Azure Key Vault.

Install:

### Package 1

```
Azure.Extensions.AspNetCore.Configuration.Secrets
```

This package provides configuration integration with Azure Key Vault Secrets.

### Package 2

```
Azure.Identity
```

This package provides Azure identity/authentication mechanisms used to authenticate the application to Azure services.

---

## 14. Add Key Vault Name to appsettings.json

We need to tell the application which Key Vault it should use.

Add:

```
{  
  "KeyVaultName": "our-first-keyvault"  
}
```

For example:

```
{  
  "ConnectionStrings": {  
    "SQLConnectionString": "Local value"  
  },  
}
```



```
"KeyVaultName": "OurFirstKeyVault"
}
```

The exact Key Vault name should be replaced with the actual Key Vault name.

---

## 15. Configure Azure Key Vault in Program.cs

Open:

```
Program.cs
```

We need to add Azure Key Vault as a configuration provider.

Conceptually:

```
builder.Configuration.AddAzureKeyVault(
    new Uri($"https://{keyVaultName}.vault.azure.net/"),
    new DefaultAzureCredential());
```

The important idea is:

```
builder.Configuration
    ↓
AddAzureKeyVault()
    ↓
Azure Key Vault
```

Now Azure Key Vault becomes one of the configuration sources for the application.

---

## 16. What Does AddAzureKeyVault Do?

When we add:

```
builder.Configuration.AddAzureKeyVault(...)
```

we are telling ASP.NET Core:

"Include Azure Key Vault as a configuration source."

Therefore, when the application asks for a configuration value, the configuration system can retrieve it from Key Vault.

---

## 17. How Does the Application Find the Secret?

Our Key Vault contains:

```
ConnectionStrings--SQLConnectionString
```

Our application asks for:

```
ConnectionStrings:SQLConnectionString
```

The Key Vault configuration provider maps:

```
ConnectionStrings--SQLConnectionString
```

to:

```
ConnectionStrings:SQLConnectionString
```

Therefore, existing application code can continue using:

```
_configuration.GetConnectionString("SQLConnectionString");
```

---

## 18. Important Advantage – No Controller Code Change

Initially:

```
var sqlConnectionString =  
    _configuration.GetConnectionString("SQLConnectionString");
```

This retrieves the value from the available configuration providers.

After adding Azure Key Vault, we don't necessarily need to change this controller code.

The configuration source changes behind the scenes.

### Before

```
Controller
  ↓
IConfiguration
  ↓
appsettings.json
  ↓
SQLConnectionString
```

### After

```
Controller
  ↓
IConfiguration
  ↓
Azure Key Vault
  ↓
ConnectionStrings--SQLConnectionString
```

The controller continues to use `IConfiguration`.

---

## 19. Authentication – Who Is Accessing Key Vault?

This is a very important concept.

The application must be authenticated and authorized to access Azure Key Vault.

In the demo, the application is running locally from Visual Studio.

The developer is logged into Azure using an account that has permission to access the Key Vault.

For example:

```
Visual Studio
  |
  ↓
```

```
Logged-in Azure User
  |
  | Permission
  ↓
Azure Key Vault
  |
  ↓
Secret
```

Therefore, `DefaultAzureCredential` can use an available developer credential to authenticate.

---

## 20. What Happens If the Application Is Deployed?

Suppose we deploy the application to:

```
Azure VM
```

The VM needs permission to access Key Vault.

We should not simply store a username and password inside the application.

Instead, we can use:

### Managed Identity

Architecture:

```
Azure VM
  |
  | Managed Identity
  ↓
Azure Key Vault
  |
  ↓
Secret
```

The same concept applies to other Azure-hosted resources that support managed identities.

---

## 21. Local Development vs Azure Deployment

### Local Development

```
Visual Studio
  |
  ↓
Developer Azure Identity
  |
  ↓
Azure Key Vault
```

### Azure Deployment

```
Azure App / VM
  |
  ↓
Managed Identity
  |
  ↓
Azure Key Vault
```

This is a very important real-world concept.

---

## 22. Role-Based Access

The identity used by the application must have appropriate permission on the Key Vault.

For example, a suitable Key Vault secret-related RBAC role can be assigned at the appropriate scope.

```
Application Identity
  |
  | RBAC Role
  ↓
Azure Key Vault
  |
  ↓
Secret
```

Always follow the **principle of least privilege**.

Give the application only the permissions it actually needs.

---

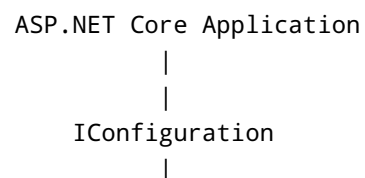
## 23. Complete Flow

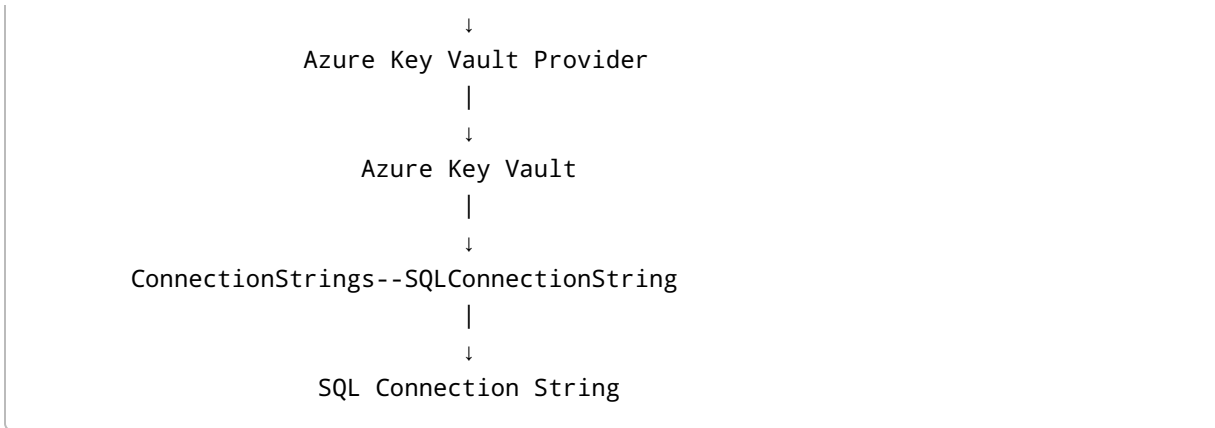
The complete process is:

1. Create ASP.NET Core Application
- ↓
2. Add connection string to appsettings.json
- ↓
3. Read connection string using IConfiguration
- ↓
4. Verify application works
- ↓
5. Create Secret in Azure Key Vault
- ↓
6. Use Key Vault naming convention:  
ConnectionStrings--SQLConnectionString
- ↓
7. Install Key Vault configuration package
- ↓
8. Install Azure.Identity
- ↓
9. Configure Key Vault in Program.cs
- ↓
10. Authenticate using Azure Identity
- ↓
11. Give application identity permission
- ↓
12. Run application
- ↓
13. IConfiguration retrieves the value from Key Vault

---

## 24. Architecture Diagram





## 25. Important Interview Questions

### Q1. Why should we use Azure Key Vault instead of appsettings.json for secrets?

Because `appsettings.json` is part of the application configuration and can potentially expose sensitive information.

Azure Key Vault provides centralized and controlled management of secrets with authentication and authorization.

### Q2. How does ASP.NET Core access Azure Key Vault?

We can add Azure Key Vault as a configuration provider using:

```
Azure.Extensions.AspNetCore.Configuration.Secrets
```

and authenticate using Azure Identity mechanisms such as:

```
DefaultAzureCredential
```

### Q3. What is DefaultAzureCredential?

`DefaultAzureCredential` is an Azure Identity credential that attempts to authenticate using an appropriate available identity/credential for the environment.

It is useful because the same authentication approach can work across different environments, such as local development and Azure-hosted applications, when the corresponding credentials/identities are configured.

---

#### Q4. Why do we use the `--` in Key Vault secret names?

Azure Key Vault secret names cannot directly represent configuration hierarchy using `:`.

Therefore:

```
ConnectionStrings--SQLConnectionString
```

can represent:

```
ConnectionStrings:SQLConnectionString
```

This allows the Key Vault configuration provider to map the secret into the ASP.NET Core configuration hierarchy.

---

#### Q5. Do we need to change the controller code after moving the secret to Key Vault?

Usually, **no**.

If the application already uses:

```
_configuration.GetConnectionString("SQLConnectionString");
```

we can keep the same code.

The configuration provider determines where the value comes from.

---

#### Q6. What happens if the application doesn't have permission to Key Vault?

The application will not be able to retrieve the secret and the Key Vault request will fail with an authorization/access error.



---

## Q7. How does an Azure application access Key Vault without storing credentials?

We can use **Managed Identity**.

Example:

```
Azure Web App
    |
    | Managed Identity
    ↓
Azure Key Vault
```

---

## 26. Important Points to Remember

### Point 1

**Secrets should not be hardcoded in application code.**

### Point 2

Azure Key Vault can securely store:

- Connection strings
- Passwords
- API keys
- Client secrets
- Other sensitive values

### Point 3

Use:

```
Azure Key Vault
+
Azure Identity
+
RBAC
```

to securely access secrets.

#### Point 4

`IConfiguration` allows application code to read configuration values without caring about the underlying configuration source.

#### Point 5

The configuration source can be:

```
appsettings.json
Environment Variables
Azure Key Vault
Other Configuration Providers
```

#### Point 6

For local development, `DefaultAzureCredential` can use the developer's available Azure credentials.

#### Point 7

For applications running in Azure, **Managed Identity** is commonly used.

---

## 27. Easy Way to Remember

```
Secret
↓
Sensitive Information

Key Vault
↓
Secure Secret Storage

IConfiguration
↓
Reads Configuration

AddAzureKeyVault()
↓
Adds Key Vault as Configuration Source

DefaultAzureCredential
↓
Authentication
```

```
Managed Identity
  ↓
Azure Resource Authentication

RBAC
  ↓
Authorization
```

---

## 28. Final Interview Answer

"In an ASP.NET Core application, instead of storing sensitive values such as database connection strings in appsettings.json, we can store them as secrets in Azure Key Vault. We add Azure Key Vault as a configuration provider and use Azure Identity for authentication. The application identity must have the required RBAC permissions on the Key Vault. Once configured, we can continue using IConfiguration, such as GetConnectionString(), without changing the business code. For local development we can use developer credentials, while Azure-hosted applications can use Managed Identity."

---

## 29. One-Minute Revision

```
ASP.NET Core
  |
  ↓
IConfiguration
  |
  ↓
AddAzureKeyVault()
  |
  ↓
Azure Key Vault
  |
  ↓
Secret
  |
  ↓
Connection String
```

**Remember:**

**Don't store secrets in code.**

**Store secrets in Key Vault.**

**Use Identity for authentication.**

**Use RBAC for authorization.**

**Use Managed Identity for Azure resources.**

**Use IConfiguration to consume the secret in the application.**